



Security Audit

Best in Slots 2nd (Blockchain Infrastructure)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	13
Audit Resources	13
Dependencies	13
Severity Definitions	14
Status Definitions	15
Audit Findings	16
Centralisation	28
Conclusion	29
Our Methodology	30
Disclaimers	32
About Hashlock	33

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Best in Slot team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Best in Slot (BiS) is the lead maintainer of the BRC20 token protocol on Bitcoin and architect of the BRC2.0 upgrade, enabling programmable Bitcoin-native assets and EVM smart contracts on Bitcoin L1. Beyond protocol stewardship, BiS powers +100 projects with comprehensive APIs for BRC20, Ordinals, and Runes. The team also builds consumer-facing products — including BiS Explorer, BiS DEX, and CoinCooker.com launchpad, driving Bitcoin's DeFi and cultural ecosystem forward.

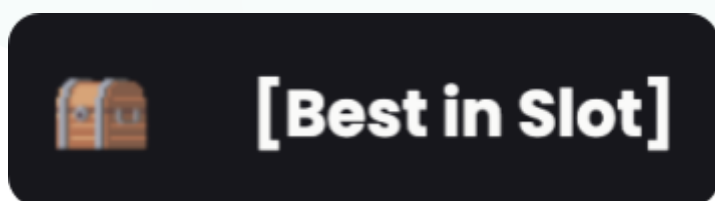
Project Name: Best in slot

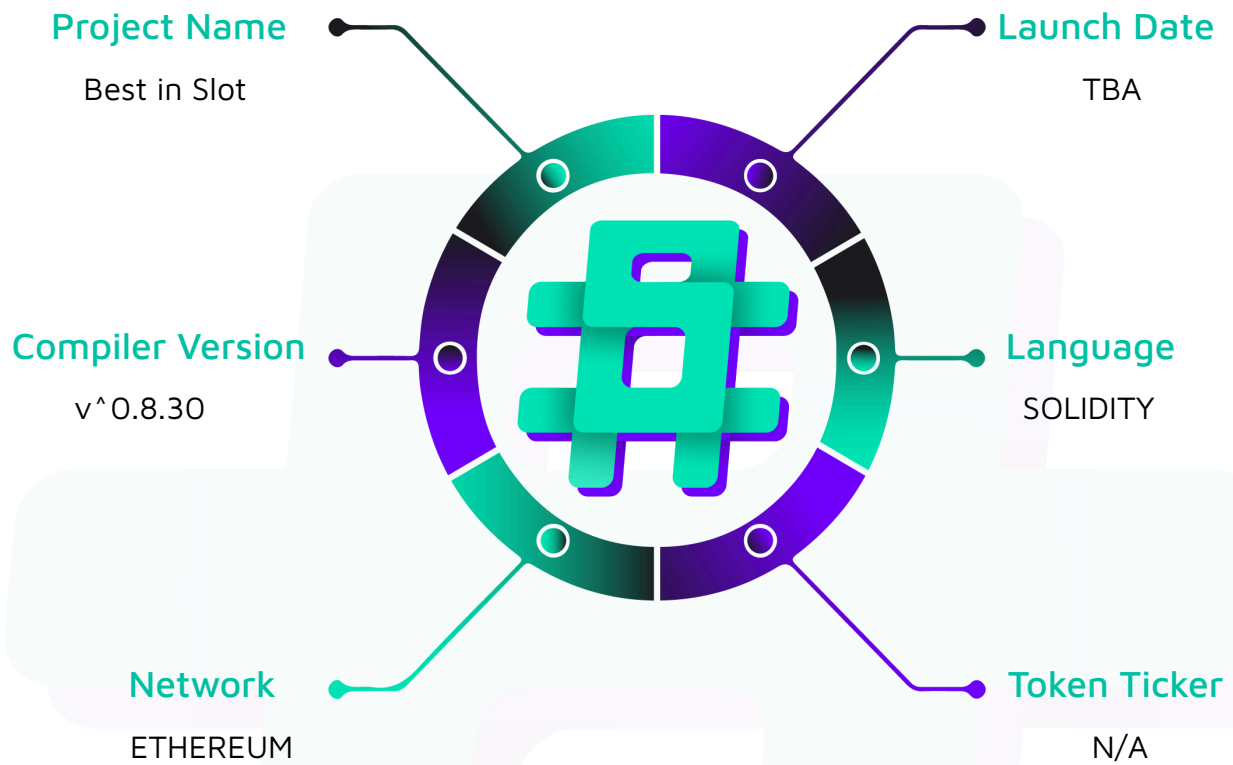
Project Type: Blockchain infrastructure

Compiler Version: ^0.8.30

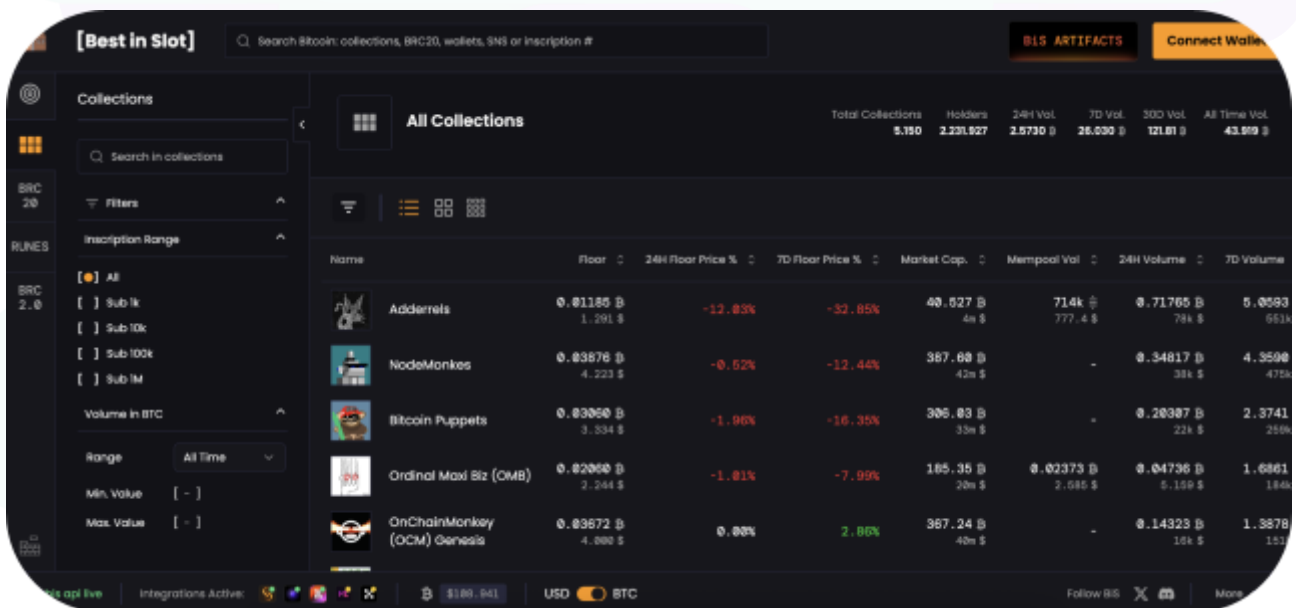
Website: <https://bestinslot.xyz/>

Logo:



Visualised Context:

Project Visuals:



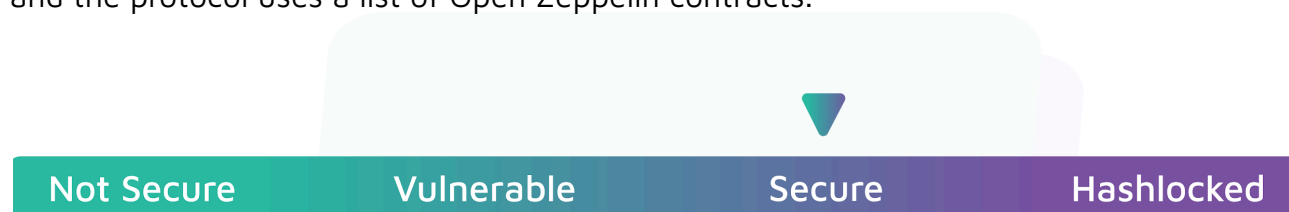
Audit Scope

We at Hashlock audited the solidity code within the Best in Slot project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Best in Slot Smart Contracts
Platform	Ethereum / Solidity
Audit Date	November, 2025
Contract 1	BiS_Swap.sol
Contract 2	bls12lib.sol
Contract 3	proxy.sol
Contract 4	timelockController.sol
Contract 5	UniswapV2Router01.sol
Contract 6	UniswapV2Factory.sol
Contract 7	UniswapV2Pair.sol
Contract 8	UniswapV2ERC20.sol
Audited GitHub Commit Hash	281a77332ffdfb703168fc49d573b28a45f3ee43
Fix Review GitHub Commit Hash	db14606e569cce914bec6eae5e69bc7ee4207afa

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved or acknowledged.

Hashlock found:

2 Medium-severity vulnerabilities

3 Low-severity vulnerabilities

1 Gas Optimisation

4 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BiS_Swap.sol - Main swap contract that enables instant swaps on UniswapV2 without front-running risks, designed for Bitcoin integration. - Key Functionality: - Deposits: Users deposit tokens with BLS-12 public keys and ECDSA signatures from a deposit signer wallet - Batch Processing: Admin wallet processes batches of operations (swaps, liquidity management, withdrawals) with aggregated BLS-12 signature verification - Operations Supported: Add/Remove liquidity on Uniswap pairs, Swap exact tokens in, Swap exact tokens out, Withdraw tokens to target addresses - Fee Management: Collects fees in wrapped BTC and optionally in token pairs - Upgradeable: Uses OpenZeppelin upgradeable contracts pattern for upgradeability	Contract achieves this functionality.
bls12lib.sol - Cryptographic library for BLS12-381 signature operations using EIP-2537 precompiles.	Contract achieves this functionality.
proxy.sol (SequencedSwapProxy) - Transparent upgradeable proxy wrapper for the BiS_Swap contract.	Contract achieves this functionality.

- Key Functionality: - Wraps OpenZeppelin TransparentUpgradeableProxy to enable upgradeable main contract - Allows admin to upgrade the implementation contract while preserving state	
timelockController.sol (SequencedSwapController) - TimeLock controller for governance operations. - Key Functionality: - Wraps OpenZeppelin's TimelockController for delayed execution of admin operations - Provides time-delayed governance for critical parameter changes	Contract achieves this functionality.
UniswapV2Router01.sol - Custom Uniswap V2 router restricted to sequencer-only operations. - Key Functionality: - Access Control: All swap and liquidity functions are onlyOwner (sequencer only) - Operations: Add liquidity to pairs, Remove liquidity from pairs, Swap exact tokens for tokens (input amount specified), Swap tokens for exact tokens (output amount specified) - Factory Management: Creates and manages Uniswap pairs through the factory - Fee Configuration: Allows setting the fee recipient address	Contract achieves this functionality.
UniswapV2Factory.sol	Contract achieves this

Factory contract for creating and managing Uniswap V2 trading pairs. Key Functionality: - Creates new token pairs using CREATE2 for deterministic addresses - Maintains mapping of all pairs - Sets the fee recipient address (only the router can call) - Access controlled by a router	functionality.
UniswapV2Pair.sol Core Uniswap V2 pair contract manages liquidity pools and swaps. Key Functionality: - Liquidity Management: Mint and burn liquidity - Swaps: Executes token swaps with constant product formula ($x * y = k$) - Reserve Management: Tracks token reserves and updates price accumulators - Fee Mechanism: Mints protocol fees to the fee recipient when enabled - Access Control: All operations are restricted to the router - Reentrancy Protection: Uses lock modifier for critical functions	Contract achieves this functionality.
UniswapV2ERC20.sol ERC20 implementation for Uniswap V2 LP (liquidity provider) tokens. - Key Functionality:	Contract achieves this functionality.

- | | |
|--|--|
| <ul style="list-style-type: none">- Standard ERC20 token functionality for LP tokens- Special Transfer Logic: Allows router to transfer from sequencer address without allowance check- Tracks sequencer and router addresses for access control | |
|--|--|

Code Quality

This audit scope involves the smart contracts of the Best in Slot project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Best in Slot project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] UniswapV2Router01#setFeeTo, UniswapV2Pair#sync, UniswapV2Pair#skim - Missing methods to call router and pair functions from sequencer

Description

The `BiS_Swap` contract lacks methods to call `UniswapV2Router01` and `UniswapV2Pair` only owner functions, preventing proper fee management and pair state synchronization.

Vulnerability Details

The `UniswapV2Router01` contract exposes a `setFeeTo()` function that updates the factory's fee recipient address. However, the `BiS_Swap` contract has its own `setFeeTo()` function that only updates the local `feeTo` variable and does not provide a method to call the router's `setFeeTo()` method.

Additionally, the `UniswapV2Pair` contract exposes `sync()` and `skim()` functions that are restricted to the owner (router). However, there are no methods in either the router or the `BiS_Swap` contracts to call these pair functions, making them inaccessible.

Impact

There is no way to update the factory's fee recipient address, synchronize pair reserves or recover tokens sent directly to pairs.

Recommendation

Add the following methods to `UniswapV2Router01`:

- `syncPair(address pair) onlyOwner`: Calls the `sync()` function on the specified pair.
- `skimPair(address pair) onlyOwner`: Calls the `skim()` function on the specified pair.

Add the following methods to `BiS_Swap`:

- `syncPair(address pair) onlyOwner`: Calls the `UniswapV2Router01.syncPair(pair)` function on the specified pair.
- `skimPair(address pair) onlyOwner`: Calls the `UniswapV2Router01.skimPair(pair)` function on the specified pair.
- `setFactoryFeeTo(address _feeTo) external onlyOwner`: Calls the `UniswapV2Router01.setFeeTo(_feeTo)` function with the specified fee recipient address. OR update the existing `BiS_Swap setFeeTo()` function to also call the router's `setFeeTo()` function.

Status

Resolved

[M-02] Bi_Swap#processBatch - Missing deadline parameter

Description

Operations in the `processBatch` function lack a deadline parameter. Without a user provided deadline, users have no guarantee about when their operation will be executed and cannot force a cancellation if execution does not happen fast enough. This also allows replay attacks when nonces overflow and reset to zero.

Vulnerability Details

The contract uses a nonce that resets to 0 when it exceeds the maximum value of the nonce. When the nonce overflows, previously signed operations could be replayed since the nonce validation would pass again.

Currently, all operations that call router functions (`_processAddLiquidity`, `_processRemoveLiquidity`, `_processSwap1`, `_processSwap2`) hardcode `block.timestamp` as the deadline parameter instead of accepting a deadline from the operation data. The `_processWithdraw` function has no deadline validation at all since it doesn't call a router function.

Impact

When nonces overflow and reset to 0, old signed operations can be replayed since there's no deadline validation to prevent execution of expired operations. Additionally, users cannot bound execution time or cancel pending operations, reducing predictability and control over their funds.

Recommendation

Add a `deadline` field to the operation data structure and validate it for all operations. The deadline should be part of the signed message by the user. For operations that call router functions, pass the deadline parameter to those functions instead of hardcoding `block.timestamp`. For the `_processWithdraw` function, add explicit deadline validation: `require(deadline >= block.timestamp, "Operation expired")`.

Status

Acknowledged

Low

[L-01] BiS_Swap#processBatch - Missing separate role for batch operations

Description

The `processBatch` function uses the `onlyOwner` modifier, requiring the same account used for sensitive administrative operations, instead of having a dedicated role for batch processing.

Vulnerability Details

The `processBatch` function is protected by `onlyOwner`, meaning it requires the same private key used for sensitive administrative functions like `setFeeTo`, `setWrappedBTCAddress`, and `setDepositSignerWallet`. Since `processBatch` is intended to be called continuously by backend systems, the owner key would likely be stored in backend infrastructure. This creates a security risk: if the backend is compromised, an attacker could obtain the owner key and gain full access to all administrative functions of the contract.

Impact

A compromise of the backend system handling batch processing could result in unauthorized access to all administrative functions, including critical operations like changing fee recipients, wrapped BTC addresses, and deposit signer wallets.

Recommendation

Create a separate role specifically for batch processing operations. Implement a new role that can be set and managed by the owner. Add a modifier `onlyBatchProcessor` and apply it to the `processBatch` function. The owner should be able to set and revoke this role using functions like `setBatchProcessor(address _batchProcessor) external onlyOwner`.

Status

Resolved

[L-02] BiS_Swap#deposit - Should use SafeERC20.forceApprove instead of direct approve

Description

Short description with proper formatting.

The `deposit` function uses direct `IERC20.approve()` instead of `SafeERC20.forceApprove()`, which may fail for tokens that do not allow changing an approval from non-zero to non-zero.

Vulnerability Details

In the `deposit()` function, the contract directly calls `approve()`. While the contract already imports and uses `SafeERC20` for other token operations like `safeTransferFrom()`, it does not use the safer `forceApprove()` method for approvals.

```
IERC20(tokenAddress).safeTransferFrom(msg.sender, address(this), amt);  
IERC20(tokenAddress).approve(address(uniswapRouter), type(uint256).max);
```

Some `ERC20` tokens, such as `USDT`, do not allow changing an approval from a non-zero value to another non-zero value. The `forceApprove` function from OpenZeppelin's `SafeERC20` library handles this, ensuring compatibility with all `ERC20` token implementations.

Impact

The direct use of `approve()` may cause transaction failures when interacting with tokens that do not support changing approvals from non-zero to non-zero values, potentially preventing deposits for certain tokens.

Recommendation

Replace the direct `approve()` call with `SafeERC20.forceApprove()`.

Status

Resolved

[L-03] UniswapV2Factory#setFeeTo - Missing event emission

Description

The `setFeeTo()` function updates the `feeTo` address state variable without emitting an event, making it difficult to track changes to this critical parameter off-chain.

Vulnerability Details

The `setFeeTo()` function in `UniswapV2Factory` modifies the `feeTo` state variable but does not emit an event to notify off-chain systems about this change.

Impact

The absence of an event makes it difficult to track and monitor changes to the `feeTo` address off-chain, reducing transparency.

Recommendation

Add an event declaration and emit it when `feeTo` is updated.

Status

Resolved

Gas

[G-01] UniswapV2Router01#sequencerAddress, UniswapV2Factory#routerAddress - State variables can be immutable

Description

The `sequencerAddress` variable in `UniswapV2Router01` and the `routerAddress` variable in `UniswapV2Factory` are set only once in the constructor and never modified afterwards, making them ideal candidates for the `immutable` keyword.

Vulnerability Details

In `UniswapV2Router01`, the `sequencerAddress` state variable is declared as a regular public variable and is assigned in the constructor. Similarly, in `UniswapV2Factory`, the `routerAddress` state variable is declared as a regular public variable and is assigned in the constructor. Neither variable is modified after the constructor execution.

Impact

Using regular state variables instead of `immutable` values that never change after deployment results in unnecessary gas costs for every read operation.

Recommendation

Declare `sequencerAddress` and `routerAddress` as `immutable`.

Status

Resolved

QA

[Q-01] proxy#SequencedSwapProxy, UniswapV2Router01#UniswapV2Router01 - Unused imports

Description

The contracts contain unused import statements that should be removed to improve code clarity and reduce compilation overhead.

Vulnerability Details

In proxy.sol, imports @openzeppelin/contracts/governance/TimelockController.sol but the TimelockController contract is never used in the SequencedSwapProxy contract.

In Uniswap-Sequencer/UniswapV2Router01.sol, imports /interfaces/IERC20.sol but the IERC20 interface is never referenced or used anywhere in the contract implementation.

Impact

Unused imports add unnecessary overhead and reduce code clarity.

Recommendation

Remove the unused import statements

Status

Resolved

[Q-02] UniswapV2ERC20 - Unused permit related code

Description

The contract declares permit-related variables and initializes the `DOMAIN_SEPARATOR` in the constructor, but these are never used since the contract does not implement a permit function.

Vulnerability Details

The `UniswapV2ERC20` contract declares three permit-related items:

- `bytes32 public DOMAIN_SEPARATOR;`
- `bytes32 public constant PERMIT_TYPEHASH`
- `mapping(address => uint) public nonces;`

The constructor initializes the `DOMAIN_SEPARATOR` using EIP-712 domain encoding, but this initialization is unnecessary since the contract does not implement a `permit` function that would use these variables.

Impact

Unused code adds unnecessary overhead and reduces code clarity.

Recommendation

Remove the unused permit-related declarations and the initialization logic from the constructor.

Status

Resolved

[Q-03] MyToken - Testing token should be moved to the test folder

Description

The MyToken contract in `src/ERC20.sol` is a testing-only contract that should be moved to a dedicated test folder.

Vulnerability Details

The MyToken contract located at `src/ERC20.sol` is explicitly marked as "for testing only" with a comment, but it is placed in the `src` directory alongside production code. Production code should be separated from testing utilities.

Impact

Test contracts in the production code directory can lead to confusion and potential accidental deployment of testing utilities.

Recommendation

Move the `ERC20.sol` contract to a `test`, `src/test`, or `src/mocks` folder.

Status

Resolved

[Q-04] BiS_Swap#processBatch - Magic numbers and repeated bit shifting logic should be extracted to constants and library functions

Description

The contract contains numerous magic numbers and repeated bit shifting operations to extract parameters from `operationDetails` that should be defined as constants and extracted into reusable library functions.

Vulnerability Details

Throughout the BiS_Swap contract, there are multiple instances of magic numbers and repeated bit shifting logic:

1. Magic numbers for operation selectors: The values `0x01`, `0x02`, `0x03`, `0x04`, and `0x05` are used directly in comparisons without named constants.
2. Magic numbers for bit positions: Hardcoded shift amounts are used throughout the code without constants defining their meaning.
3. Magic numbers for bit masks: Hex masks are repeated across multiple functions without named constants.
4. Magic number for basis points: The value `10000` is used for fee calculations without a constant.
5. Repeated bit shifting logic: The same bit shifting patterns to extract parameters from `operationDetails` are duplicated across multiple functions:
 - a. `pkIdx` extraction: `uint256(operationDetails) >> 200`
 - b. `opSelector` extraction: `uint8((uint256(operationDetails) >> 192) & 0xff)`
 - c. `pkNonce` extraction: `uint32((uint256(operationDetails) >> 56) & 0xffffffff)`
 - d. `token1FeeBps` extraction: `(uint256(operationDetails) >> 40) & 0xffff`
 - e. `token2FeeBps` extraction: `(uint256(operationDetails) >> 24) & 0xffff`
 - f. `fee` extraction: `uint256(operationDetails) & 0x00ffffff`
 - g. `tick1Idx` and `tick2Idx` extraction: `(uint256(operationDetails) >> 160) & 0xffffffff` and `(uint256(operationDetails) >> 128) & 0xffffffff`
 - h. `tickCnt` extraction: `uint8((uint256(operationDetails) >> 184) & 0xff)`

- i. Dynamic `tickIdx` extraction: `(uint256(operationDetails) >> (184 - 32 * (i + 1))) & 0xffffffff`

This duplication makes the code harder to maintain, increases the risk of errors when modifying the bit layout, and reduces code readability.

Impact

Magic numbers and repeated bit shifting logic reduce code maintainability, increase the risk of errors when modifying the operation details layout, and make the codebase harder to understand and audit.

Recommendation

Create a library (e.g., `OperationDetailsLib`) that contains:

1. Named constants for all magic numbers (operation selectors, bit positions, masks, basis points)
2. Pure internal functions to extract each parameter from `operationDetails` (e.g., `getPkIdx(bytes32 operationDetails)`, `getOpSelector(bytes32 operationDetails)`, `getPkNonce(bytes32 operationDetails)`, etc.)
3. Use the library in `BiS_Swap` with `using OperationDetailsLib for bytes32` to enable method style calls like `operationDetails.getPkIdx()`, `operationDetails.getOpSelector()`, `operationDetails.getPkNonce()`, etc.

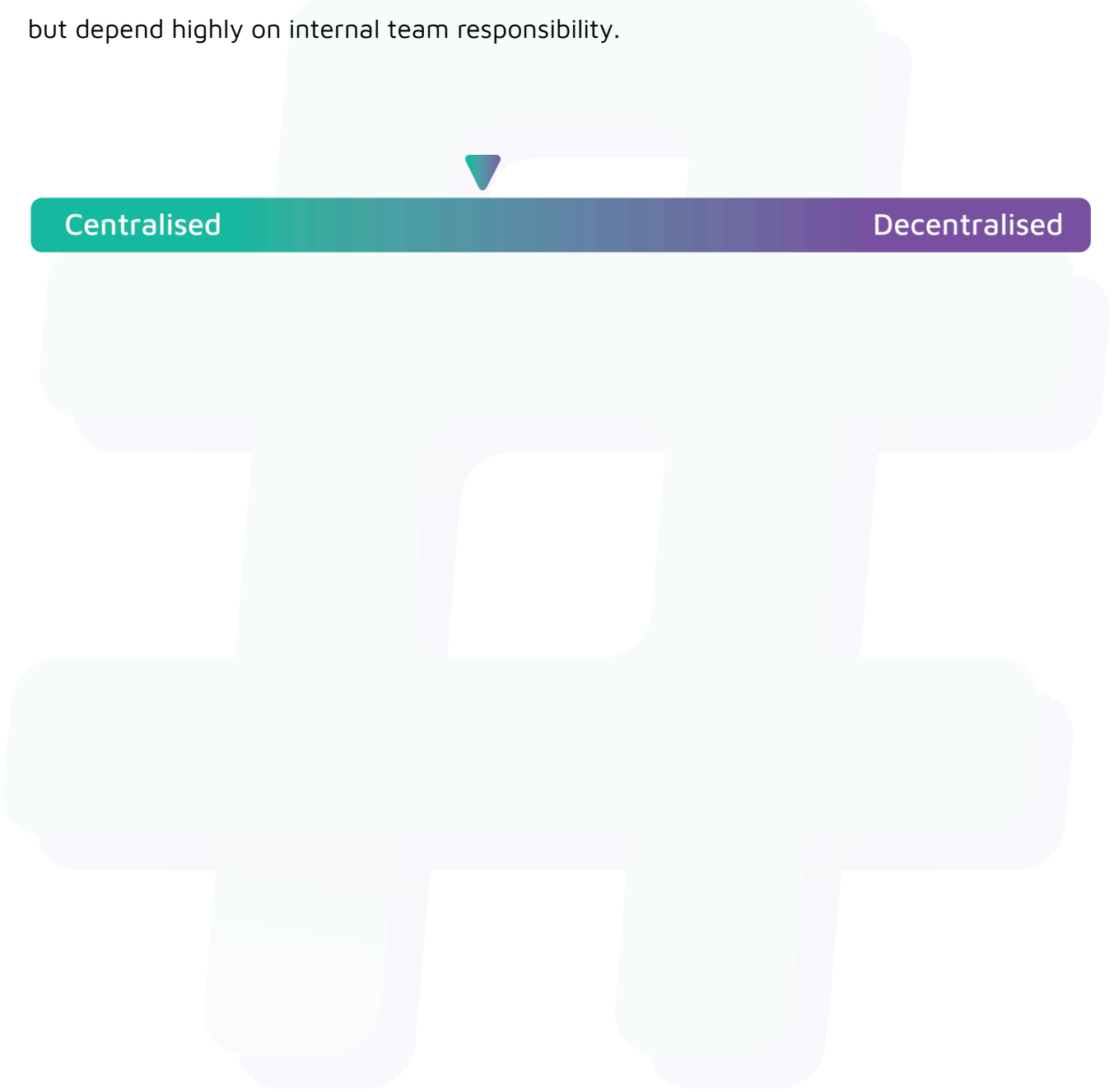
Status

Resolved

Centralisation

The Best in Slot project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Best in Slot project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved or acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.